Відокремлений структурний підрозділ

«Фаховий коледж інформаційних технологій»

Національного університету «Львівська політехніка»



лабораторної роботи

з дисципліни « WEB-програмування »

Виконав:

Студент групи КН-43

Гарбіч Богдан Назарович

Перевірив:

викладач

Роман Скалоцький

Львів 2026

ЗВІТ

з лабораторної роботи №7

Тема: Завантаження даних з публічного API та відображення на сторінці

1. Мета роботи

Ознайомитися з принципами роботи REST API, поняттям endpoint та асинхронним завантаженням даних у вебзастосунках. Опанувати виконання HTTP GET-запитів за допомогою функції `fetch()` з використанням синтаксису `async/await`, обробку об'єктів `Response` (`response.ok`, `response.status`), серіалізацію даних з JSON у масиви JavaScript (`response.json()`) та інтеграцію динамічних даних у DOM-структуру навчального проєкту «Акустична варта».

2. Постановка задачі

1. Створити та налаштувати сервіс MockAPI з ресурсом `sensors` та підготувати JSON-масив тестових даних.
2. Розробити модуль `services/api.js` з асинхронною функцією `fetchSensorsFromAPI()`, що запитує дані з MockAPI.
3. Інтегрувати отримання даних із сервера у головний файл `app.js` та забезпечити збереження зв'язку з ООР-архітектурою (`NoiseSensor`, `MonitoringSystem`), розробленою в ЛР №5.
4. Оновити функцію рендерингу карток сенсорів та обробники фільтрів за районом і статусом з ЛР №6.
5. Налаштувати обробку можливих мережевих або серверних помилок за допомогою конструкцій `try...catch` із виведенням повідомлення користувачеві.
6. Перевірити виконання HTTP GET-запитів у вкладці DevTools → Network.

3. Структура REST API (MockAPI)

Взаємодія з тестовим сервером здійснюється через наступний endpoint:

- **Base URL:** https://6af1a6e542c6561ced387919.mockapi.io/api/v1
- **Resource Endpoint:** https://6af1a6e542c6561ced387919.mockapi.io/api/v1/sensors
- **HTTP Method:** `GET`

Приклад JSON-структури відповіді сервера:


```
[
  {
    "id": "1",
    "name": "1",
    "district": "Галицький",
    "location": "просп. Свободи, 14",
    "noiseLevel": 72,
    "status": "critical"
  },
  {
    "id": "2",
    "name": "2",
    "district": "Сихівський",
    "location": "вул. Зелена, 102",
    "noiseLevel": 52,
    "status": "normal"
  },
  {
    "id": "3",
    "name": "3",
    "district": "Франківський",
    "location": "вул. Наукова, 8",
    "noiseLevel": 68,
    "status": "warning"
  },
  {
    "id": "4",
    "name": "4",
    "district": "Шевченківський",
    "location": "просп. Чорновола, 21",
    "noiseLevel": 83,
    "status": "critical"
  }
]
```

4. Програмна реалізація

4.1. Модуль запитів до API (`js/services/api.js`)

```
const API_URL = "https://6af1a6e542c6561ced387919.mockapi.io/api/v1/sensors";

/**
 * Виконує HTTP GET-запит до MockAPI для отримання переліку сенсорів
 * @returns {Promise<Array>} Масив об'єктів сенсорів
 */
export async function fetchSensorsFromAPI() {
    try {
        const response = await fetch(API_URL);

        if (!response.ok) {
            throw new Error(`Помилка сервера: ${response.status}
${response.statusText}`);
        }

        const sensors = await response.json();
        return sensors;
    } catch (error) {
        console.error("Не вдалося завантажити сенсори з API:",
error);
        throw error;
    }
}
```

4.2. Головний файл застосунку (`js/app.js`)

```
import { NoiseSensor } from './models/NoiseSensor.js';
import { MonitoringSystem } from './services/MonitoringSystem.js';
import { fetchSensorsFromAPI } from './services/api.js';

// DOM-елементи
const sensorList = document.querySelector("#sensors-list") ||
document.querySelector(".sensor-list");
const districtFilter = document.querySelector("#district-filter");
const criticalFilter = document.querySelector("#critical-filter");

const monitoringSystem = new MonitoringSystem();
```

```
// Асинхронна ініціалізація застосунку
async function initApp() {
   try {
        const rawData = await fetchSensorsFromAPI();

        monitoringSystem.sensors = [];
        rawData.forEach(item => {
            const sensor = NoiseSensor.fromObject(item);
            monitoringSystem.addSensor(sensor);
        });

        renderSensors(monitoringSystem.sensors);
        console.log("Дані успішно завантажено з API:", rawData);

    } catch (error) {
        if (sensorList) {
            sensorList.innerHTML = `<p style="color: red; text-
align: center; grid-column: 1 / -1;">Не вдалося завантажити дані з
сервера. Перевірте з'єднання.</p>`;
        }
    }
}

// Створення картки сенсора
function createSensorCard(sensor) {
    const status = sensor.getStatus ? sensor.getStatus() :
(sensor.status || "normal");

    const card = document.createElement("article");
    card.classList.add("sensor-card", `sensor-card--${status}`);

    const header = document.createElement("div");
    header.classList.add("sensor-card-header");

    const title = document.createElement("h3");
    title.textContent = sensor.name || sensor.id;

    const statusBadge = document.createElement("span");
    statusBadge.classList.add("sensor-status-badge");
    statusBadge.textContent = status;

    header.append(title, statusBadge);
```

```
    const district = document.createElement("p");
    district.classList.add("sensor-info-item");
    district.innerHTML = `📍 Район: <span class="label-
val">${sensor.district} район</span>`;

    const location = document.createElement("p");
    location.classList.add("sensor-info-item");
    location.innerHTML = `🏢 Локація: <span class="label-
val">${sensor.location || "Не вказано"}</span>`;

    const level = document.createElement("p");
    level.classList.add("sensor-info-item", "sensor-noise-level");
    level.innerHTML = `📊 Рівень шуму: <span class="label-
val">${sensor.noiseLevel} дБ</span>`;

    card.append(header, district, location, level);
    return card;
}

// Рендеринг списку карток
function renderSensors(sensorArray) {
    if (!sensorList) return;
    sensorList.replaceChildren();

    if (sensorArray.length === 0) {
        sensorList.innerHTML = `<p style="grid-column: 1 / -1; text-
align: center;">Сенсорів за такими критеріями не знайдено.</p>`;
        return;
    }

    sensorArray.forEach((sensor) => {
        const card = createSensorCard(sensor);
        sensorList.append(card);
    });
}

// Фільтрація даних
function applyFilters() {
    const selectedDistrict = districtFilter ? districtFilter.value :
"all";
    const criticalOnly = criticalFilter ? criticalFilter.checked :
```

```
false;

    const filteredSensors = monitoringSystem.sensors.filter((sensor)
=> {
        const matchesDistrict = (selectedDistrict === "all" ||
sensor.district === selectedDistrict);
        const status = sensor.getStatus ? sensor.getStatus() :
sensor.status;
        const matchesStatus = (!criticalOnly || status ===
"critical");

        return matchesDistrict && matchesStatus;
    });

    renderSensors(filteredSensors);
}

if (districtFilter) districtFilter.addEventListener("change",
applyFilters);
if (criticalFilter) criticalFilter.addEventListener("change",
applyFilters);

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});
```

5. Результати виконання та перевірки

1. **Мережеві запити (DevTools Network):** Під час підняття локального вебсервера через Python (`python -m http.server 8000`) та відкриття сторінки браузер виконує асинхронний GET-запит до MockAPI. У DevTools підтверджено status code `200 OK`.
2. **Динамічне відображення:** Вміст контейнера `.sensor-list` успішно генерується з даних, завантажених з сервера. На картках коректно відображаються рівні шуму (52, 68, 72, 83 дБ) та статуси (`normal`, `warning`, `critical`).
3. **Обробка помилок:** При некоректній адресі API або відсутності мережевого з'єднання спрацьовує блок `catch`, а у DOM виводиться повідомлення: *"Не вдалося завантажити дані з сервера. Перевірте з'єднання."*

6. Відповіді на контрольні запитання

1. **Що таке API?**

API (Application Programming Interface) — це набір готових методів і протоколів, через які одна програма (наприклад, frontend у браузері) взаємодіє та обмінюється даними з іншою (backend або зовнішнім сервером).

2. **Що таке endpoint?**

Endpoint — це конкретна URL-адреса точки доступу до ресурсів API (наприклад, `/api/v1/sensors`), за якою клієнт відправляє HTTP-запити.

3. **Для чого використовується HTTP GET?**

Для отримання (читання) даних із сервера без зміни стану бази даних.

4. **Що повертає `fetch()`?**

Функція `fetch()` повертає `Promise`, який при успішному виконанні повертає об'єкт `Response`.

5. **Що знаходиться у змінній `response` після `await fetch()`?**

Екземпляр об'єкта `Response`, який містить метадані HTTP-відповіді (заголовки, статус-код `status`, булевий прапорець `ok`), але ще не розпарсене тіло відповіді.

6. **Для чого використовується `response.json()`?**

Для зчитування тіла відповіді та його асинхронного перетворення з текстового формату JSON у структуровані об'єкти/масиви JavaScript.

7. **Чому в типовому GET-запиті використовуються два `await`?**

Перший `await` чекає завершення мережевої доставки HTTP-відповіді від сервера (`fetch`), а другий `await` чекає повного зчитування та парсингу JSON-тіла (`response.json()`).

8. **Що означає `response.ok`?**

Булева властивість, яка дорівнює `true`, якщо HTTP-код відповіді належить до діапазону 200–299 (Успіх).

9. **Де у DevTools можна побачити HTTP-запит?**

У вкладці **Network** (Мережа) підфільтр `Fetch/XHR`.

10. Для чого потрібен `try/catch`?

Для безпечного відловлювання та обробки виняткових ситуацій (мережеві збої, відсутність інтернету, помилки форматування), щоб уникнути фатального зупинення виконання скрипта.

7. Висновки

Під час виконання лабораторної роботи було успішно інтегровано зовнішній REST API service (MockAPI) у вебпроєкт «Акустична варта». Опановано роботу з асинхронним JavaScript (`async/await`, `fetch`), перетворення JSON у структуровані класи ES6, обробку помилок мережі та динамічний рендеринг завантажених даних у DOM.